

Linear Haskell

Practical Linearity in a Higher-Order Polymorphic Language

OLI BURGER¹

¹liolin@liolin.ch

Zurich, 2026-09-21

Papers We LoveSM $f(x)=x$

Oli, Master Student at OST

Interested in

- Compilers
- Programming Language Design
- Type Systems
- Formal Methods

Currently doing

- Teaching a Compiler Construction Course
- Working on [rust-lang/a-mir-formality](#)

Overview

1. Motivation
2. Linear Haskell
3. Core Calculus

MOTIVATION

The Haskell Programming Language

Haskell is a **pure** and **lazy functional** programming language.

pure

no side effects

lazy

do as little work as possible, as late as possible

functional

programming with values

If it compiles, it works!

File IO in Haskell

Assume we want to implement a simple Haskell application.

Read the first line from a file and return it.

```
1  openFile  :: FilePath -> IO Handle
2  closeFile :: Handle  -> IO ()
3  readLine  :: Handle  -> IO String
4
5  firstLine :: FilePath -> IO String
6  firstLine fp =
7      do { f <- openFile fp
8          ; closeFile f
9          ; line <- readLine f
10         ; return line }
```

 Haskell

Is this implementation correct?

Major Bug in `firstLine`

```
1  openFile  :: FilePath -> IO Handle
```

 Haskell

```
2  closeFile :: Handle -> IO ()
```

```
3  readLine  :: Handle -> IO String
```

```
4
```

```
5  firstLine :: FilePath -> IO String
```

```
6  firstLine fp =
```

```
7    do { f <- openFile fp
```

```
8        ; closeFile f
```

```
9        ; line <- readLine f
```

```
10       ; return line }
```

} (1) `f` is used after it was invalidated

Conclusion

Bugs similar to use-after-free in C!

And this in a language that has the reputation: if it compiles, it works!

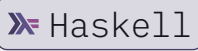
LINEAR HASKELL

Intuition for Linear Haskell

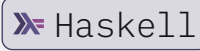
In a typical type system, we can use the arguments 0, 1 or many times.

In a linear system: A function is linear if it uses its arguments **exactly** once!

```
1 data Coin
2 data Water
3 data Soda
```



```
1 water :: Coin -> Water
2 soda  :: Coin -> Soda
3 -- I can use the coin multiple times ???
4 buyingWaterAndSoda :: Coin -> (Water, Soda)
5 buyingWaterAndSoda c = (water c, soda c)
```

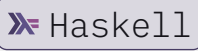


Intuition for Linear Haskell

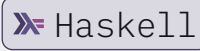
In a typical type system, we can use the arguments 0, 1 or many times.

In a linear system: A function is linear if it uses its arguments **exactly** once!

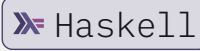
```
1 data Coin
2 data Water
3 data Soda
```



```
1 water :: Coin -> Water
2 soda  :: Coin -> Soda
3 -- I can use the coin multiple times ???
4 buyingWaterAndSoda :: Coin -> (Water, Soda)
5 buyingWaterAndSoda c = (water c, soda c)
```



```
1 water' :: Coin → Water
2 soda'  :: Coin → Soda
3
4 buyingWaterAndSoda' :: Coin → (Water, Soda)
5 buyingWaterAndSoda' c = (water c, soda c) -- Error!
```



Linear Functions: The Linear Function Type

The normal (unrestricted) function type: $\rightarrow$

The linear function type: $\multimap$

```
1 -- non-linear
2 f :: s -> t
3 -- linear
4 g :: s  $\multimap$  t
5 -- mixed
6 h :: s  $\multimap$  t -> u
```

 Haskell

The type of g guarantees that, if g is used exactly once, the argument s is used exactly once. The type of h guarantees that, s is only used once, but t can be used arbitrarily.

Linear Functions: Examples

```
1  -- valid
2  double :: Int -> Int
3  double x = x + x
4
5  -- invalid, x is used twice
6  double2 :: Int → Int
7  double2 x = x + x
8
9  -- valid
10 double3 :: Int → Int
11 double3 x = x * 2
12
13 -- valid
14 double (double3 2) == 8
```

 Haskell

Linear Functions: What does “exactly once” mean?

- To consume a value of atomic base type (like `Int` or `Ptr`) exactly once, just evaluate it.
- To consume a function value exactly once, apply it to one argument, and consume its result exactly once
- To consume a pair exactly once, pattern-match on it, and consume each component exactly once
- In general, to consume a value of an algebraic datatype exactly once, pattern-match on it, and consume all its linear components exactly once.

Linear Data Types: Construction

```
1 (,) :: a -> b -> (a,b)
```

 Haskell

```
2 (,) :: a  $\multimap$  b  $\multimap$  (a,b)
```

```
3
```

```
4 value :: (Int, Int)
```

```
5 value = (,) 0 1
```

Which type should we give the `(,)` data constructor?

Linear Data Types: Construction

```
1 (,) :: a -> b -> (a,b)
2 (,) :: a → b → (a,b)
3
4 value :: (Int, Int)
5 value = (,) 0 1
```

» Haskell

Which type should we give the `(,)` data constructor?

All datatype defined using Haskell-98 (non-GADT) syntax are linear constructor functions.

```
1 (,) :: a → b → (a, b)
2 (:) :: a → [a] → [a]
3 Just :: a → Maybe a
```

» Haskell

Linear Data Types: Pattern-Matching

Although the construction is linear, the resulting value can be used arbitrarily.

```
1  (,) :: a → b → (a, b)
2
3  f1 :: (Int, Int) -> (Int, Int)
4  f1 x = case x of (a, b) -> (a, a)
5
6  f2 :: (Int, Int) → (Int, Int)
7  f2 x = case x of (a, b) -> (b, a)
8
9  f1 (1, 0) == (1, 1)
10 f2 (1, 0) == (0, 1)
```

 Haskell

Linear Data Types: Non-Linearity

What if we want to construct a data type, where its component is not required to be used linear?

```
1  -- GADT-Syntax
2  data Unrestricted a where { Unrestricted :: a -> Unrestricted a }
3
4  f :: Unrestricted Int -> (Int, Int)
5  f (Unrestricted i) = (i, i)
6
7  writeTwice :: (Handle, String) -> IO Handle
8  writeTwice (f, s) = write f (s ++ s)
9
10 writeTwice' :: (Handle, Unrestricted String) -> IO Handle
11 writeTwice' (f, Unrestricted s) = write f (s ++ s)
```

 Haskell

Linear Data Types: Non-Linearity

What if we want to construct a data type, where its component is not required to be used linear?

```
1  -- GADT-Syntax
2  data Unrestricted a where { Unrestricted :: a -> Unrestricted a }
3
4  f :: Unrestricted Int → (Int, Int)
5  f (Unrestricted i) = (i, i)
6
7  writeTwice :: (Handle, String) → IO Handle
8  writeTwice (f, s) = write f (s ++ s)
9
10 writeTwice' :: (Handle, Unrestricted String) → IO Handle
11 writeTwice' (f, Unrestricted s) = write f (s ++ s)
```

 Haskell

Linear Data Types: Non-Linearity

What if we want to construct a data type, where its component is not required to be used linear?

```
1  -- GADT-Syntax
2  data Unrestricted a where { Unrestricted :: a -> Unrestricted a }
3
4  f :: Unrestricted Int -> (Int, Int)
5  f (Unrestricted i) = (i, i)
6
7  writeTwice :: (Handle, String) -> IO Handle
8  writeTwice (f, s) = write f (s ++ s)
9
10 writeTwice' :: (Handle, Unrestricted String) -> IO Handle
11 writeTwice' (f, Unrestricted s) = write f (s ++ s)
```

 Haskell

Linear Data Types: Non-Linearity

What if we want to construct a data type, where its component is not required to be used linear?

```
1  -- GADT-Syntax
2  data Unrestricted a where { Unrestricted :: a -> Unrestricted a }
3
4  f :: Unrestricted Int → (Int, Int)
5  f (Unrestricted i) = (i, i)
6
7  writeTwice :: (Handle, String) → IO Handle
8  writeTwice (f, s) = write f (s ++ s)
9
10 writeTwice' :: (Handle, Unrestricted String) → IO Handle
11 writeTwice' (f, Unrestricted s) = write f (s ++ s)
```

 Haskell

The map Function

```
1 map f []      = []
2 map f (x:xs) = f x : map f xs
```

» Haskell

Which type should `map` have?

- $(a \rightarrow b) \rightarrow [a] \rightarrow [b]$
- $(a \multimap b) \rightarrow [a] \multimap [b]$

```
1 -- non-linear
2 f :: [Int] -> [Int]
3 f xs = map (\x -> x + x) xs
4
5 -- linear
6 g :: [Int] -> [Int]
7 g xs = map (\x -> x + 1) xs
```

» Haskell

Multiplicity Polymorphism

The type of `map` in Linear Haskell is:

$$\text{map} :: \forall p. (a \rightarrow_p b) \rightarrow [a] \rightarrow_p [b]$$
$$\text{map} :: (a \text{ \%m } \text{->} b) \text{->} [a] \text{ \%m } \text{->} [b]$$

`map` is a multiplicity polymorphic function!

Linear IO

There are no linear types, only linear functions!

But `IO` is a type, that represents IO computation – How can we make it linear?

Linear IO

There are no linear types, only linear functions!

But IO is a type, that represents IO computation – How can we make it linear?

Make the type generic over multiplicity and control the continuation.

```
1 type IO_L p a
2 return :: a %p -> IO_L p a
3 bind    :: IO_L p a → (a %p -> IO_L q b) → IO_L q b
4
5 openFile :: FilePath → IO_L 1 Handle
6 readLine :: Handle → IO_L 1 (Handle, Unrestricted String)
7 closeFile :: Handle → IO_L ω ()
```

 Haskell

Linear IO

There are no linear types, only linear functions!

But IO is a type, that represents IO computation – How can we make it linear?

Make the type generic over multiplicity and control the continuation.

```
1 type IO_L p a
2 return :: a %p -> IO_L p a
3 bind    :: IO_L p a → (a %p -> IO_L q b) → IO_L q b
4
5 openFile :: FilePath → IO_L 1 Handle
6 readLine :: Handle → IO_L 1 (Handle, Unrestricted String)
7 closeFile :: Handle → IO_L ω ()
```

 Haskell

Linear IO

There are no linear types, only linear functions!

But IO is a type, that represents IO computation – How can we make it linear?

Make the type generic over multiplicity and control the continuation.

```
1 type IO_L p a
2 return :: a %p -> IO_L p a
3 bind    :: IO_L p a → (a %p -> IO_L q b) → IO_L q b
4
5 openFile :: FilePath → IO_L 1 Handle
6 readLine :: Handle → IO_L 1 (Handle, Unrestricted String)
7 closeFile :: Handle → IO_L ω ()
```

 Haskell

Linear IO

There are no linear types, only linear functions!

But `IO` is a type, that represents IO computation – How can we make it linear?

Make the type generic over multiplicity and control the continuation.

```
1 type IO_L p a
2 return :: a %p -> IO_L p a
3 bind    :: IO_L p a → (a %p -> IO_L q b) → IO_L q b
4
5 openFile :: FilePath → IO_L 1 Handle
6 readLine :: Handle → IO_L 1 (Handle, Unrestricted String)
7 closeFile :: Handle → IO_L ω ()
```

 Haskell

How to Solve the Initial Problems?

```
1  openFile  :: FilePath → IO_L 1 Handle
2  closeFile :: Handle → IO_L ω ()
3  readLine  :: Handle → IO_L 1 (Handle, Unrestricted String)
4
5  firstLine :: FilePath → IO String
6  firstLine fp = do
7    { f <- openFile fp;
8      ; (f, Unrestricted line) <- readLine f
9      ; closeFile f
10     ; return line
11    }
```

 Haskell

How to Solve the Initial Problems?

```
1  openFile  :: FilePath → IO_L 1 Handle
2  closeFile :: Handle → IO_L ω ()
3  readLine  :: Handle → IO_L 1 (Handle, Unrestricted String)
4
5  firstLine :: FilePath → IO String
6  firstLine fp = do
7    { f <- openFile fp;
8      ; (f, Unrestricted line) <- readLine f
9      ; closeFile f
10     ; return line
11    }
```

 Haskell

How to Solve the Initial Problems?

```
1  openFile  :: FilePath → IO_L 1 Handle
2  closeFile :: Handle → IO_L ω ()
3  readLine  :: Handle → IO_L 1 (Handle, Unrestricted String)
4
5  firstLine :: FilePath → IO String
6  firstLine fp = do
7    { f <- openFile fp;
8      ; (f, Unrestricted line) <- readLine f
9      ; closeFile f
10     ; return line
11    }
```

 Haskell

How to Solve the Initial Problems?

```
1  openFile  :: FilePath → IO_L 1 Handle
2  closeFile :: Handle → IO_L ω ()
3  readLine  :: Handle → IO_L 1 (Handle, Unrestricted String)
4
5  firstLine :: FilePath → IO String
6  firstLine fp = do
7    { f <- openFile fp;
8      ; (f, Unrestricted line) <- readLine f
9      ; closeFile f
10     ; return line
11    }
```

 Haskell

Quick Note Regarding Strictness

Linear does not imply strict!

Remember

If (λx) is consumed exactly once, then each component of the pair is consumed exactly once.

```
1 f :: a → (a, Bool)
2 f x = (x, True)
3
4 snd :: (a, b) → b
5
6 -- >>> snd (f undefined) == True
```

 Haskell

But `snd` does **NOT** consume its argument exactly once.

CORE CALCULUS

Core Calculus $\lambda_{\rightarrow}^q$

GHC compiles Haskell to a IL called GHC Core: a simply-typed λ -calculus.

Linear Haskell itself is not formalised, but a core calculus $\lambda_{\rightarrow}^q$.

The Syntax of $\lambda_{\rightarrow}^q$

Multiplicities

$$\pi, \mu := 1 \mid \omega \mid p \mid \pi + \mu \mid \pi \cdot \mu$$

Types

$$A, B := A \rightarrow_{\pi} B \mid \forall p. A \mid D \ p_1 \dots p_n$$

Contexts

$$\Gamma, \Delta := (x :_{\mu} A), \Gamma \mid \sim$$

Datatype declaration

$$\text{data } D \ p_1 \dots p_n \text{ where } \left(c_k : A_1 \rightarrow_{\pi_1} \dots A_{n_k} \rightarrow_{\pi_{n_k}} D \right)_{k=1}^m$$

The Syntax of $\lambda_{\rightarrow}^q$

Terms

$e, s, t, u := x$	variable
$\lambda_{\pi}(x : A).t$	abstraction
$t s$	application
$\lambda p.t$	multiplicity abstraction
$t \pi$	multiplicity application
$c t_1 \dots t_n$	data construction
$\text{case}_{\pi} t \text{ of } \{c_k x_1 \dots x_{n_k} \rightarrow u_k\}_{k=1}^m$	case
$\text{let}_{\pi} x_1 : A_1 = t_1 \dots x_n : A_n = t_n \text{ in } u$	let

Static Semantics

$$\begin{array}{c}
\frac{}{\omega\Gamma + x :_1 A \vdash x : A} \text{var} \qquad \frac{\Gamma, x :_\pi A \vdash t : B}{\Gamma \vdash \lambda_\pi(x : A).t : A \rightarrow_q B} \text{abs} \qquad \frac{\Gamma \vdash t : A \rightarrow_\pi B \quad \Delta \vdash u : A}{\Gamma + \pi\Delta \vdash t u : B} \text{app} \\
\\
\frac{\Delta_i \vdash t_i : A_i \quad c_k : A_1 \rightarrow_{\mu_1} \dots \rightarrow_{\mu_{n-1}} A_n \rightarrow_{\mu_n} D \ p_1 \dots p_n \text{ constructor}}{\omega\Gamma + \sum_i \mu_i[\pi_1 \dots \pi_n] \Delta_i \vdash c_k t_1 \dots t_n : D \ \pi_1 \dots \pi_n} \text{con} \\
\\
\frac{\Gamma \vdash t : D \ \pi_1 \dots \pi_n \quad \Delta, x_1 :_{\pi\mu_i[\pi_1 \dots \pi_n]} A_i, \dots, x_{n_k} :_{\pi\mu_{n_k}[\pi_1 \dots \pi_n]} A_{n_k} \vdash u_k : C \quad \text{for each } c_k : A_1 \rightarrow_{\mu_1} \dots \rightarrow_{\mu_{n_k-1}} A_{n_k} \rightarrow_{\mu_{n_k}} D \ p_1 \dots p_n}{\pi\Gamma + \Delta \vdash \text{case}_\pi t \text{ of } \{c_k x_1 \dots x_{n_k} \rightarrow u_k\}_{k=1}^m} \text{case} \\
\\
\frac{\Gamma_i \vdash t_i : A_i \quad \Delta, x_1 :_\pi A_1 \dots x_n :_\pi A_n \vdash u : C}{\Delta + \pi \sum_i \Gamma_i \vdash \text{let}_\pi x_1 : A_1 = t_1 \dots x_n : A_n = t_n \text{ in } u : C} \text{let} \qquad \frac{\Gamma \vdash t : A \quad p \text{ fresh for } \Gamma}{\Gamma \vdash \lambda p.t : \forall p.A} \text{m.abs} \\
\\
\frac{\Gamma \vdash t : \forall p.A}{\Gamma \vdash t \ \pi : A[\pi/p]} \text{m.app}
\end{array}$$

101 of Judgment Rules

1. If we find a proof for all premises, we have a proof for the conclusion.
2. Given the left-hand side of $\vdash$, we can deduce the right hand side.

$$\frac{\text{Premise 1} \quad \dots \quad \text{Premise } n}{\text{Conclusion}} \text{ the name}$$

$$\frac{\Gamma, x :_{\pi} A \vdash t : B}{\Gamma \vdash \lambda_{\pi}(x : A).t : A \rightarrow_q B} \text{abs}$$

Abstraction

$$\frac{\Gamma, x :_{\pi} A \vdash t : B}{\Gamma \vdash \lambda_{\pi}(x : A).t : A \rightarrow_q B} \text{abs}$$

To prove that there is an abstraction from A to B with body t , show that, there exists a term $t : B$ under the hypothesis Γ supplemented with $x :_{\pi} A$

Application

$$\frac{\Gamma \vdash t : A \rightarrow_{\pi} B \quad \Delta \vdash u : A}{\Gamma + \pi \Delta \vdash t u : B} \text{app}$$

- There exists an abstraction $t : A \rightarrow_{\pi} B$ with multiplicity π in the environment Γ
- There exists a value $u : A$ in the environment Δ

Then we can deduce that the application of t to u yields a value of type B

Assume $t : A \rightarrow_1 B$

Then all free variables in u must be used once

Assume $t : A \rightarrow_{\omega} B$

Then all free variables in u must be used with multiplicity ω

This is expressed by *scaling* the context: $\pi \Delta$

Context Addition and Scaling

Addition

$$(x :_{\pi} A, \Gamma) + (x :_{\mu} A, \Delta) = x :_{\pi+\mu} A, (\Gamma + \Delta)$$

$$(x :_{\pi} A, \Gamma) + \Delta = x :_{\pi} A, \Gamma + \Delta \quad (x \notin \Delta)$$

$$() + \Delta = \Delta$$

Scaling

$$\pi(x :_{\mu} A, \Gamma) = x :_{\pi\mu} A, \pi\Gamma$$

$+$ and $\cdot$ are associative and commutative

$$(\pi + \mu) + 1 = \pi + (\mu + 1)$$

$$\pi + \mu = \mu + \pi$$

1 is the unit of $\cdot$

$$1 \cdot p = p$$

$\cdot$ distributes over $+$

$$\pi \cdot (q + \mu) = \pi \cdot q + \pi \cdot \mu$$

$$\omega \cdot \omega = \omega$$

$$1 + 1 = 1 + \omega = \omega + \omega = \omega$$

Let

$$\frac{\Gamma_i \vdash t_i : A_i \quad \Delta, x_1 :_{\pi} A_1 \dots x_n :_{\pi} A_n \vdash u : C}{\Delta + \pi \sum_i \Gamma_i \vdash \text{let}_{\pi} x_1 : A_1 = t_1 \dots x_n : A_n = t_n \text{ in } u : C} \text{let}$$

- There exists a bunch of term $t : A$ in various environments
- Under the assumption Δ supplemented with various bindings, we can deduce a term $u : C$

Then we can deduce, there is a valid let binding.

We explicitly annotated the let binding with the multiplicity. It tells us how often the right hand side is consumed.

Var

$$\frac{}{\omega\Gamma + x :_1 A \vdash x : A} \text{var}$$

Adding a variable to the context, does not change the type of the variable.

The rule contains also *weakening*: ignoring of not used variables.

By scaling the context with ω enables weakening.

Multiplicity Abstraction and Application

$$\frac{\Gamma \vdash t : A \quad p \text{ fresh for } \Gamma}{\Gamma \vdash \lambda p. t : \forall p. A} \text{m.abs}$$

- There exists a term $t : A$
- There is a fresh multiplicity variable in Γ

Then we can create a term that is generic over multiplicity

$$\frac{\Gamma \vdash t : \forall p. A}{\Gamma \vdash t \pi : A[\pi/p]} \text{m.app}$$

- There exists a term $t : A$ which is generic over multiplicity

Applying a concrete multiplicity yields a non-generic variant

Construction

$$\frac{\Delta_i \vdash t_i : A_i \quad c_k : A_1 \rightarrow_{\mu_1} \dots \rightarrow_{\mu_{n-1}} A_n \rightarrow_{\mu_n} D \ p_1 \dots p_n \text{ constructor}}{\omega\Gamma + \sum_i \mu_i[\pi_1 \dots \pi_n] \Delta_i \vdash c_k \ t_1 \dots t_n : D \ \pi_1 \dots \pi_n} \text{con}$$

- There exists a bunch of terms $t : A$ in various environments
- There exists a bunch of constructors to build a type $D \ p_1 \dots p_2$

Using a constructor function yields a value of type $D \ \pi_1 \dots \pi_2$

- $\omega\Gamma$: weakening in Γ
- $\mu_i[\pi_1 \dots \pi_n]$: Replace the multiplicity variable p_i in μ with the multiplicity π_i from the list.

Premise

data $D \ p_1 \ p_2$ where $\{ c : A \rightarrow_{p_1} B \rightarrow_{p_2} D \}$

$a :_1 A \in \Delta_1$

$b :_\omega B \in \Delta_2$

Conclusion

$c \ (a : A) \ (b : B) : D \ 1 \ \omega$

Case Expressions

$$\frac{\begin{array}{c} \Gamma \vdash t : D \pi_1 \dots \pi_n \quad \Delta, x_1 :_{\pi \mu_1 [\pi_1 \dots \pi_n]} A_1, \dots, x_{n_k} :_{\pi \mu_{n_k} [\pi_1 \dots \pi_n]} A_{n_k} \vdash u_k : C \\ \text{for each } c_k : A_1 \rightarrow_{\mu_1} \dots \rightarrow_{\mu_{n_k-1}} A_{n_k} \rightarrow_{\mu_{n_k}} D p_1 \dots p_n \end{array}}{\pi \Gamma + \Delta \vdash \text{case}_{\pi} t \text{ of } \{c_k x_1 \dots x_{n_k} \rightarrow u_k\}_{k=1}^m} \text{case}$$

- There exists a term $t : D \pi_1 \dots \pi_n$
- There exists a bunch of bindings $x :_{\pi} A$ that allow us to build a term $u : C$
- There are multiple constructor functions to build a type $D : p_1 \dots p_2$

Then we can conclude, there is a valid case expression with pattern matching that yields a value of type C

Premise

Conclusion

$\text{data } D p_1 p_2 \text{ where } \{c : A \rightarrow_{p_1} B \rightarrow_{p_2} D p_1 p_2\}$

$\text{case}_1 t \text{ of } \{c a b \rightarrow c a b_2 D 1 1\}$

$c : D 1 \omega \in \Gamma$

$a :_1 A$

$b_1 :_{\omega} B$

$b_2 :_1 B$

The paper contains more:

- complete semantics and proofs for soundness
- case studies
- related and future work

Papers We LoveSM *$f(x)=x$*